



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 1
по курсу «Разработка параллельных и распределенных
программ»
«Распараллеливание алгоритма вычисления произведения двух
матриц»

Студент группы ИУ9-51Б Григорьева С. А.

Преподаватель Царев А. С.

Москва 2025

1 Задача

Две квадратные матрицы A и B размерности n сначала перемножить стандартным алгоритмом. Замерить время вычисления, сравнить с временем при вычислении элементов матрицы C не по строкам, а по столбцам. Размер матриц подобрать таким образом, чтобы время выполнения на вашей машине было не слишком непоказательно малым (меньше нескольких минут), но и не чересчур большим (несколько часов). Использовать библиотечные функции для вычисления произведений матриц нельзя.

Затем конечную матрицу C условно разделить на примерно равные прямоугольные подматрицы и распараллелить программу таким образом, чтобы каждый поток занимался вычислением своей подматрицы. Матрицы A и B для этого разделить на примерно равные группы строк и столбцов соответственно. Сделать для разного количества потоков (разных разбиений), также замерить время вычисления, сравнить с вычислениями стандартным алгоритмом. Также по окончании вычислений сравнивать получившуюся матрицу с той, что была вычислена стандартным алгоритмом, для проверки правильности вычислений (проверка во время выполнения задачи не входит).

Уделить внимание проблеме синхронизации потоков: основной поток, который создает потоки, занимающиеся вычислением подматриц, не должен закончиться до того, как они закончат свою работу, поэтому организовать ожидание и сигнализацию завершения рабочих потоков.

2 Практическая реализация

Исходный код программы представлен в листингах 1– 2– 3.

Листинг 1 — часть 1

```

1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "runtime"
7     "sync"
8     "time"
9 )
10
11 type Matrix [][] float64
12 const N = 2000
13
14 func getNewMatrix(n int) Matrix {
15     m := make(Matrix, n)
16     for i := range m {
17         m[i] = make([]float64, n)
18     }
19     return m
20 }
21
22 func getRandomMatrix(n int, r *rand.Rand) Matrix {
23     m := getNewMatrix(n)
24     for i := 0; i < n; i++ {
25         for j := 0; j < n; j++ {
26             m[i][j] = r.Float64()*10 - 5
27         }
28     }
29     return m
30 }
31
32 func isEqual(a, b Matrix, eps float64) bool {
33     if len(a) != len(b) {
34         return false
35     }
36     n := len(a)
37     for i := 0; i < n; i++ {
38         for j := 0; j < n; j++ {
39             if diff := a[i][j] - b[i][j]; diff > eps || diff < -eps {
40                 return false
41             }
42         }
43     }
44     return true
45 }
46
47 func rowMul(a, b Matrix) Matrix {
48     n := len(a)
49     c := getNewMatrix(n)
50     for i := 0; i < n; i++ {
51         for k := 0; k < n; k++ {
52             aik := a[i][k]
53             for j := 0; j < n; j++ {
54                 c[i][j] += aik * b[k][j]
55             }
56         }
57     }
58     return c
59 }

```

Листинг 2 — часть 2

```

1 func columnMul(a, b Matrix) Matrix {
2     n := len(a)
3     c := getNewMatrix(n)
4     for j := 0; j < n; j++ {
5         for i := 0; i < n; i++ {
6             sum := 0.0
7             for k := 0; k < n; k++ {
8                 sum += a[i][k] * b[k][j]
9             }
10            c[i][j] = sum
11        }
12    }
13    return c
14 }
15
16 func parallelMu(a, b Matrix, blocks int) Matrix {
17     n := len(a)
18     c := getNewMatrix(n)
19
20     if blocks < 1 {
21         blocks = 1
22     }
23     if blocks > n {
24         blocks = n
25     }
26
27     var wg sync.WaitGroup
28     rowsPerBlock := n / blocks
29     extra := n % blocks
30
31     startRow := 0
32     for bIdx := 0; bIdx < blocks; bIdx++ {
33         blockRows := rowsPerBlock
34         if bIdx < extra {
35             blockRows++
36         }
37         endRow := startRow + blockRows
38         if startRow >= endRow {
39             break
40         }
41
42         wg.Add(1)
43         go func(r0, r1 int) {
44             defer wg.Done()
45             for i := r0; i < r1; i++ {
46                 for k := 0; k < n; k++ {
47                     aik := a[i][k]
48                     for j := 0; j < n; j++ {
49                         c[i][j] += aik * b[k][j]
50                     }
51                 }
52             }
53         }(startRow, endRow)
54         startRow = endRow
55     }
56     wg.Wait()
57     return c
58 }

```

Листинг 3 — часть 3

```
1 func main() {  
2     r := rand.New(rand.NewSource(time.Now().UnixNano()))  
3  
4     A := getRandomMatrix(N, r)  
5     B := getRandomMatrix(N, r)  
6  
7     t1 := time.Now()  
8     CRow := rowMul(A, B)  
9     durRow := time.Since(t1)  
10  
11    t2 := time.Now()  
12    CCol := columnMul(A, B)  
13    durCol := time.Since(t2)  
14  
15    blocks := runtime.NumCPU()  
16    t3 := time.Now()  
17    CPar := parallelMu(A, B, blocks)  
18    durPar := time.Since(t3)  
19    fmt.Println()  
20    fmt.Printf("Row-wise time: %v\n", durRow)  
21    fmt.Printf("Column-wise time: %v\n", durCol)  
22    fmt.Printf("Parallel computation time (%d blocks): %v\n", blocks  
    , durPar)  
23 }
```

3 Характеристики устройства

Модель оборудования: MEGABOOK T14TA (TECNO T14TA)

Процессор: 11th Gen Intel(R) Core(TM) i5-115

Память: 16 Гиб

Емкость диска: 512 Гб

4 Время работы программы

Время работы умножения по столбцам: 39.120307253 s

Время работы умножения по строкам: 7.401064948 s

Время вычисления при 2 потоках: 4.655073141 s

Время вычисления при 4 потоках: 2.450579983 s

Время вычисления при 8 потоках: 2.343974272 s

